

2026-5-20

多系统稳定性分析讲解

姓名 李昌全

掌锐电子

目录

一，多系统稳定性问题有哪些	2
1.1 从现象角度看	2
1.2 从问题种类看	2
二，多系统稳定性问题如何分析	3
2.1 KE/HWT 引发系统重新卡死分析	4
2.1.1 gat 工具解析 db 文件	4
2.2.2 mtk gat 工具中 gdb 使用	4
2.2.3 NE/KE analyze	7
2.2 系统无法启动分析	10
2.2.1 串口 log 分析	10
2.2.2 回读解析 expdb 分区	14
三，案例讲解 15	15
3.1 kernel 踩内存	15
3.2 user 版本 Android 无法启动	17

一，多系统稳定性问题有哪些

1.1 从现象角度看

1，无法启动

需要从串口 log 入手查找原因。

2，无法关机

从 mobilelog 分析关机流程。

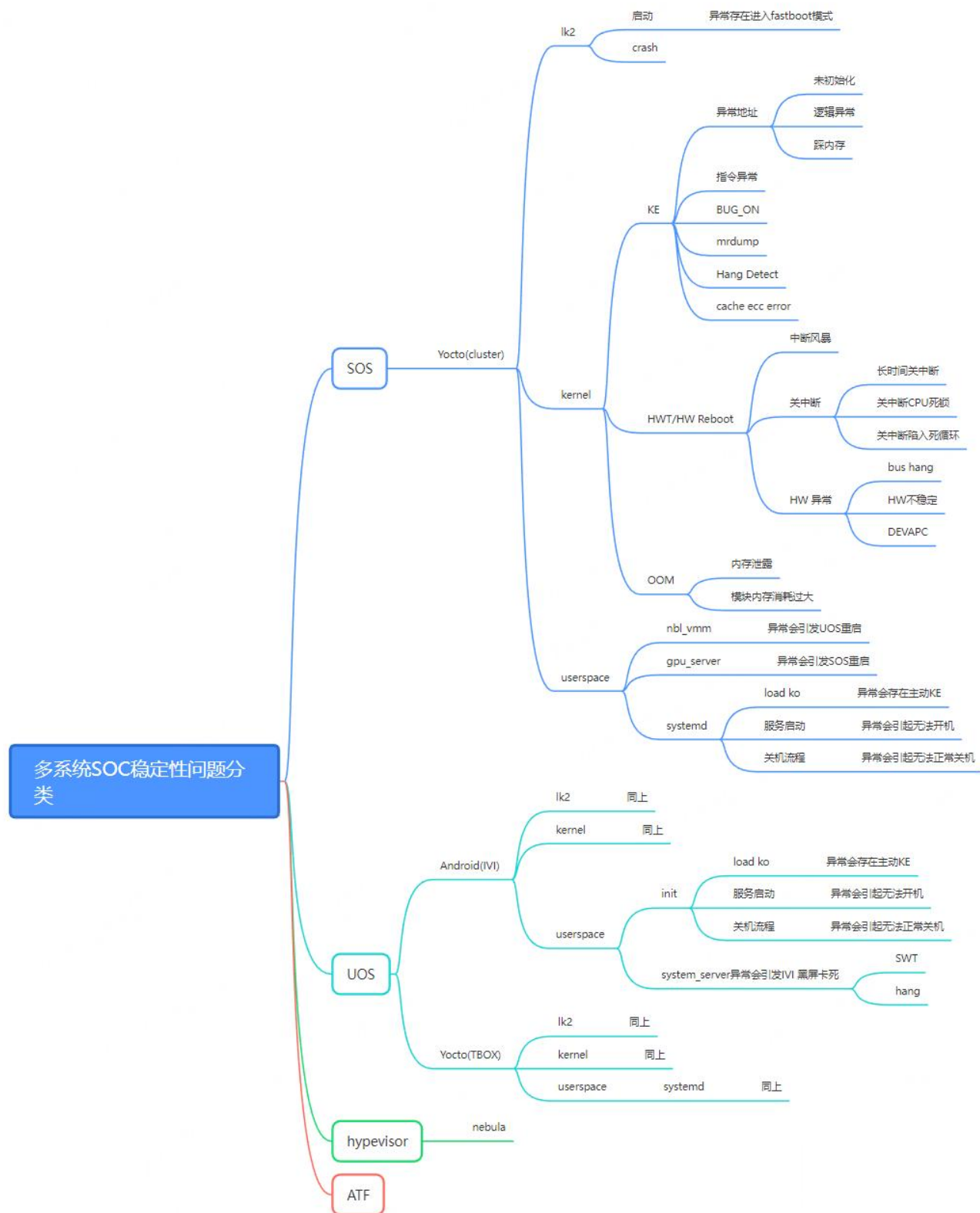
3，卡死

优先检查 db 文件，若无则通过 mobilelog 找到卡死时间点。

4，重启

优先检查 db 文件，若无则通过 mobilelog 查找重启原因及其时间点。

1.2 从问题种类看

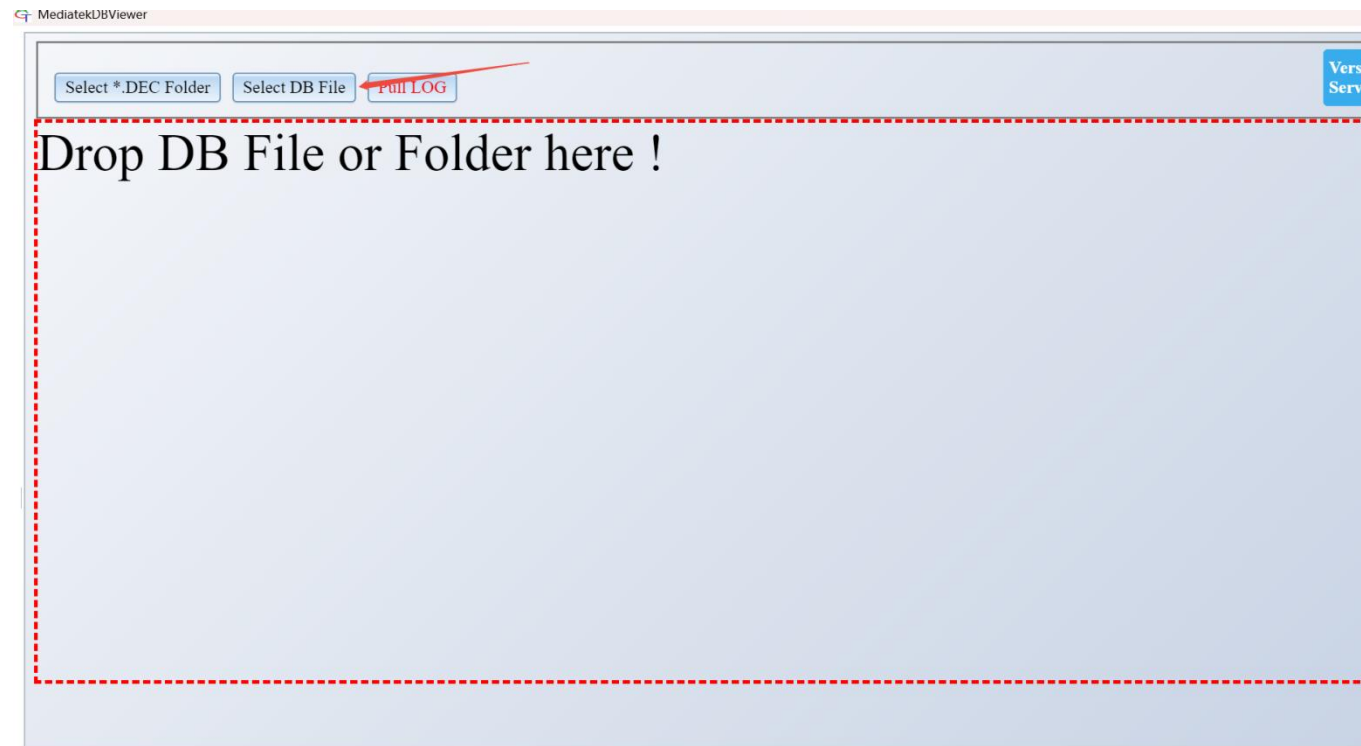


二，多系统稳定性问题如何分析

2.1 KE/HWT 引发系统重新卡死分析

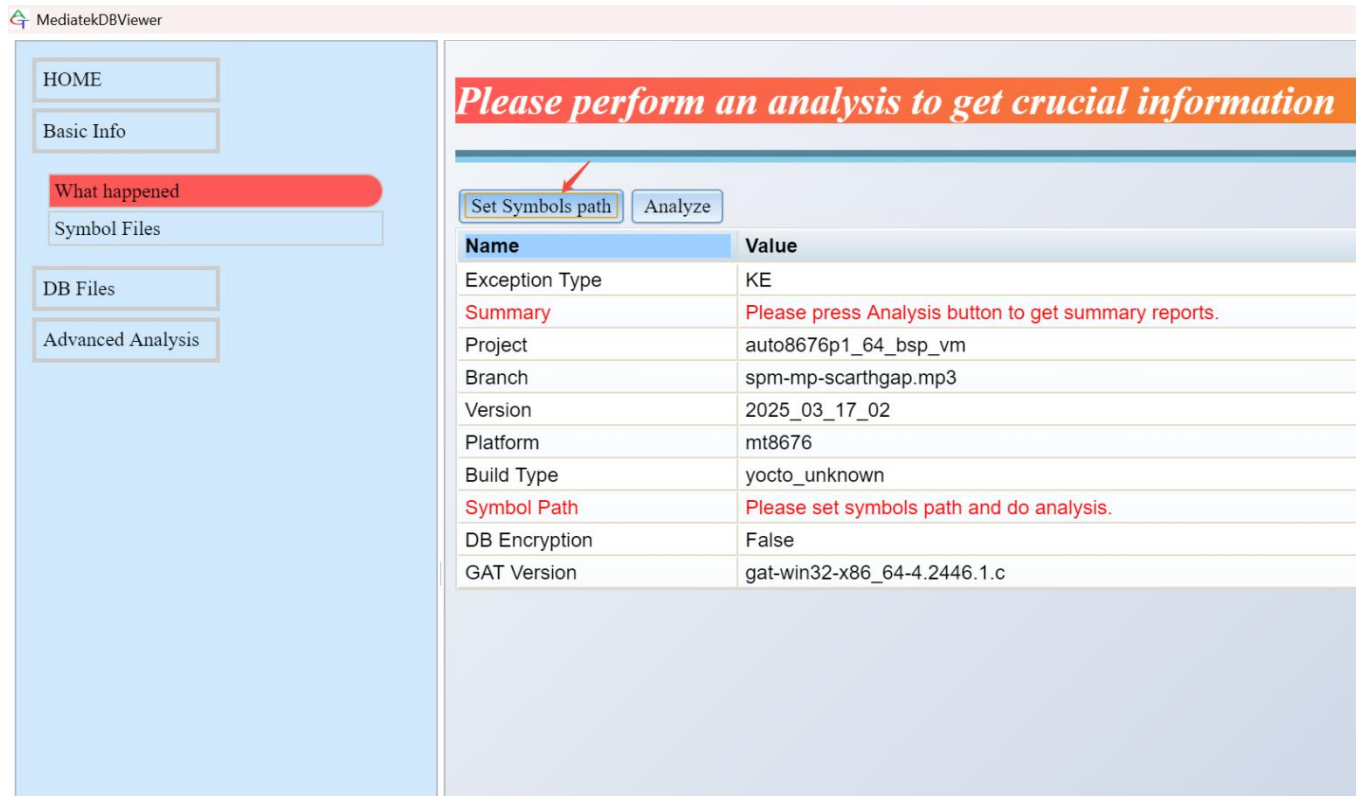
2.1.1 gat 工具解析 db 文件

1. 加入文件之后 db_fatal.00.KE.dbg, 点击 start



2.2.2 mtk gat 工具中 gdb 使用

1. 然后这个界面设置下 symbols 目录, 然后点击 Analyze



2. 待它解析完成后，点击左上角的 Advanced Analysis, 选择启动 gdb

The screenshot displays the MEDIALABVIEWER application interface. On the left, a sidebar contains navigation buttons: HOME, Basic Info, DB Files, and Advanced Analysis (highlighted with a red box). Below these are three buttons: extract kconfig, Launch GDB(SYS_MINI_RDUMP) (indicated by a red arrow), and Decode HANG_DETECT_RAW. The main area on the right is titled 'Exception report' in a red header. It includes two buttons: 'Set Symbols path' and 'Reanalyze'. Below these is a table with two columns: 'Name' and 'Value'. The table contains the following data:

Name	Value
Exception Type	KE
Exception Sub Type	DATA_ABORT
Summary	...
Last Analysis	20250524-164617
Project	auto8676p1_64_bsp_vm
Branch	spm-mp-scarthgap.mp3
Version	2025_03_17_02
Platform	mt8676
Build Type	yocto_unknown
Symbol Path	D:\LOG\gdb\temp
Reference	...
DB Encryption	False
GAT Version	gat-win32-x86_64-4.2446.1.c

Below the table is a section titled 'What_happened' containing a log table:

time	log
2025-0524 16:46:17	D:\LOG\gdb\temp\pub_data\00.RE.gdbg.DE
2025-0524 16:46:17	memory pressure analyzer failed.
2025-0524 16:46:17	error: Failed to analyze memory pressure
2025-0524 16:46:17	Input the file of keyword is not exist,and
2025-0524 16:46:17	Input log type is NULL
2025-0524 16:46:17	Do not belong to any type!

3. 此时的 gdb 会自动加载 symbols 文件，如下图所示：

```

D:\tools\gat-win32-x86_64-4.2 x + v

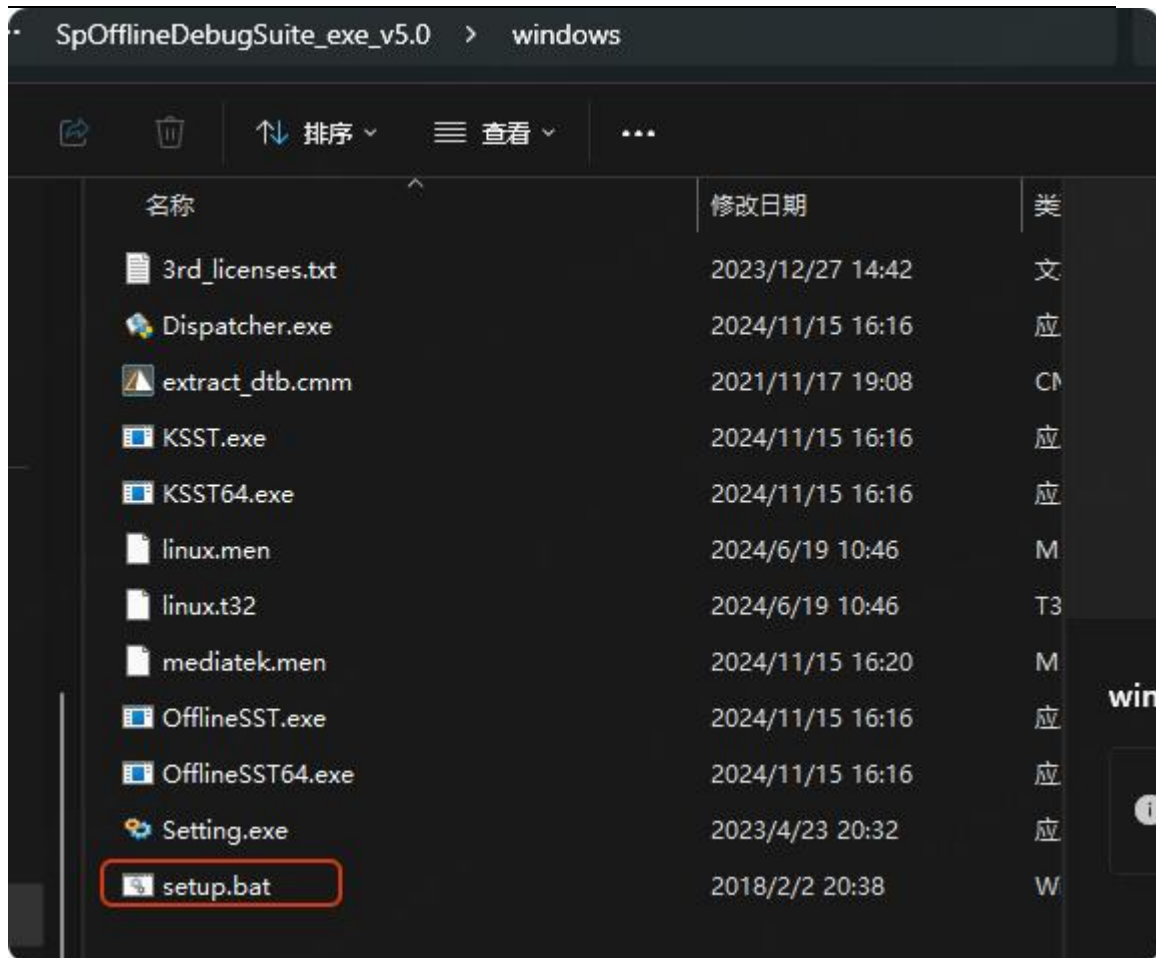
.hyp.data..percpu_addr = 0xfffffffffe204f0c000
.hyp.reloc_addr = 0xfffffffffe204f0df50
.rela.dyn_addr = 0xfffffffffe204f0e548
.data_addr = 0xfffffffffe205210000
__bug_table_addr = 0xfffffffffe20543df90
.hyp.data_addr = 0xfffffffffe2054620f8
.mmuoff.data.write_addr = 0xfffffffffe205464000
.mmuoff.data.read_addr = 0xfffffffffe205464800
.pecoff_edata_padding_addr = 0xfffffffffe205464808
.bss_addr = 0xfffffffffe205465000
warning: core file may not match specified executable file.
[New LWP 103]
[New LWP 100]
[New LWP 101]
[New LWP 102]
[New LWP 104]
[New LWP 105]
[New LWP 106]
[New LWP 107]
Core was generated by `proc_cmdline
#0 __arm_smccc_smc () at /usr/src/kernel/arch/arm64/kernel/smccc-call.S:61
warning: 61 /usr/src/kernel/arch/arm64/kernel/smccc-call.S: No such file or directory
[Current thread is 1 (LWP 103)]
add symbol table from file "D:/LOG/gdb/temp/mrdump.ko" at
.text_addr = 0xfffffffffe2031b5000
.init.text_addr = 0xfffffffffe2031d2000
add symbol table from file "D:/LOG/gdb/temp/iommu_debug.ko" at
.text_addr = 0xfffffffffe2056e7000
.init.text_addr = 0xfffffffffe20571a000
add symbol table from file "D:/LOG/gdb/temp/mtk_iommu.ko" at
.text_addr = 0xfffffffffe205729000
.init.text_addr = 0xfffffffffe20574a000
add symbol table from file "D:/LOG/gdb/temp/bcm-xgbe.ko" at
.text_addr = 0xfffffffffe208dae000
.init.text_addr = 0xfffffffffe208def000
(gdb) |

```

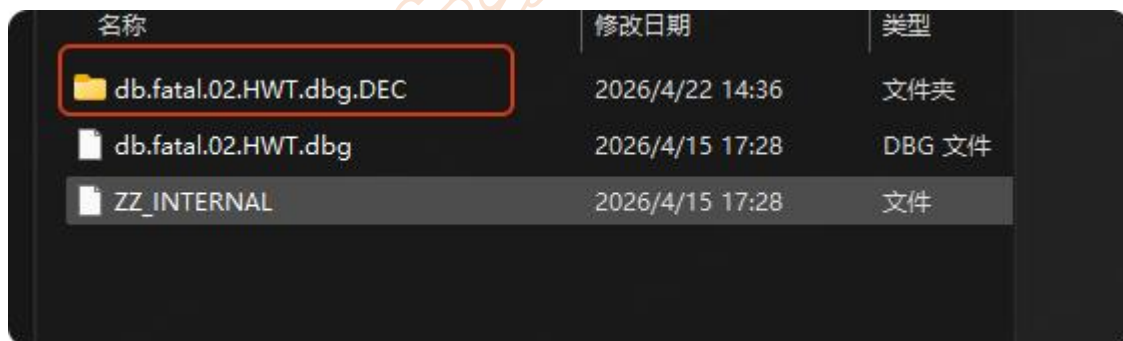
2.2.3 NE/KE analyze

NE/KE analyze 工具可以直接分析 gat 解析出来的数据，并提供分析报告。

1, 先要安装 Sp0fflineDebugSuite_exe



2, 找到 gat 解析出来的文件夹



3, 进行 NE/KE analyze

右键 gat 解析出来的文件夹，选择菜单栏中 “NE/KE analyze”，会进行分析生成常用分析数据如下：

名称	修改日期	类型	大小
scrutils	2026/4/22 14:36	文件夹	
src	2026/4/22 14:36	文件夹	
symbols	2026/4/22 14:36	文件夹	
debug.cmm	2026/4/22 14:36	CMM 文件	27 KB
DFD_SOC.dfd		DFD 文件	28,672 KB
out.json	2026/4/22 14:35	JSON 文件	2 KB
_exp_main.txt		文本文档	1 KB
_extract.log	2026/4/22 14:35	文本文档	1 KB
_分析报告.txt	2026/4/22 14:36	文本文档	4 KB
project_info.txt	2026/4/22 14:35	文本文档	1 KB
CMDQ_DEBUG		文件	581 KB
CMDQ_RECORD		文件	16 KB
CMDQ_STATUS		文件	1 KB
INTERRUPT_NUM_NAME_MAPS		文件	37 KB

名称	修改日期	类型	大小
 SYS_GPU_LOG_ALL		文件	2,048 KB
 SYS_GPU_MFGSYS_LATCH_LOG		文件	15 KB
 SYS_GPUEB_COREDUMP		文件	641 KB
 SYS_GPUEB_LAST_LOG		文件	12 KB
 SYS_GPUEB_XFILE		文件	487 KB
 SYS_GPU_FREQ_STATUS_BOOTUP		文件	2 KB
 SYS_HANG_DETECT_RAW		文件	4,096 KB
 SYS_HWT_IRQ_RAW		文件	128 KB
 SYS_HYPER_MINI_RDUMP		文件	624 KB
 SYS_KERNEL_LOG	2026/4/22 14:36	文件	4,604 KB
 SYS_KERNEL_LOG_DATA_RAW		文件	4,096 KB
 SYS_KERNEL_LOG_DESCS_RAW		文件	3,072 KB
 SYS_KERNEL_LOG_INFOS_RAW		文件	11,264 KB
 SYS_KERNEL_LOG_TP_RAW		文件	1 KB
 SYS_KERNEL_NOTES		文件	1 KB
 SYS_LAST_CPU_BUS		文件	5 KB
 SYS_LAST_DRAM		文件	1 KB
 SYS_LAST_DVFSRC_DATA		文件	3 KB
 SYS_LAST_KMSG	2026/4/22 14:36	文件	259 KB
 SYS_LAST_MMQOS_RAW		文件	4 KB

2.2 系统无法启动分析

2.2.1 串口 log 分析

当系统无法启动到 adb 阶段，无法正常导出日志，需要抓取串口日志，即从串口 log 入手，分析系统启动各个阶段状态。

多系统所有阶段的 log 都会输出到同一个串口，很容易混淆，可借助 log 关键词进行标识，方便 debug。

2.2.1.1 1k 日志

1k 的每句 log 是没有关键词进行区分的，只能通过 start 和 end log 进行辨识。

起始 log，多系统的 1k 关键词都可以通过

welcome to lk2

进行标识，顺序 sos lk->android uos lk -> tbox uos lk

LLA 会搜索到三次“**welcome to lk2**”

```

Search "welcome to lk2" (3 hits in 1 file)
D:\work_tools\sscom\ReceivedTofile-COM15-2025_1_2_10-43-48.DAT (3 hits)
Line 506: welcome to lk2
Line 16509: welcome to lk2
Line 24705: welcome to lk2

```

下面会有 **project:**关键词 区分不同系统，如下：

1, SOS

lk variant: BL2

boot args 0xbbbbbbb64 0x80000000 0x80000000 0x1001c010

version:

arch: arm64

platform: mediatek

target: auto8676p1_64_ufs

project: auto8676p1_64_hyp

buildid: 202512260254_

2, android

lk variant: BL33

boot args 0x0 0x0 0x0 0x0

version:

arch: arm64

platform: mediatek

target: auto8676p1_64_ufs

project: auto8676p1_64_hyp-an

buildid: 202512260254_

3, tbox

lk variant: BL33

boot args 0x0 0x0 0x0 0x0

version:

arch: arm64

platform: mediatek

target: auto8676p1_64_ufs

project: auto8676p1_64_uos_tbox

buildid: 202512181932_

结束 log，会有所不一样，如下：

1, sos lk 结束 log

Turn off BL31 console

2, android uos lk 结束 log

lk finished --> jump to linux kernel 64Bit

3, tbox uos lk 结束 log

LK run time

2.2.1.2 hypevisor 串口 log

可以确定是 hypevisor log 关键词

ARM boot EL2

通过这句 log，再可以通过关键词

nebula_log_driver

搜索确定周围 hypervisor uart log

```
[ 7.957324][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957327][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957329][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957332][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957335][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957337][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957340][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957343][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957346][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957349][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957352][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957356][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957359][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957368][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957370][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957373][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957376][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957379][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957382][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957385][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957388][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957391][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957394][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957398][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957399][T1600867] kworker/ul6:4: nebula_ipc_driver virtio0: [name:nebula_ipc
[ 7.957401][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957405][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957408][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957410][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957413][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957416][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
[ 7.957417][T1601576] systemd-modules: grt-trusty-virtio nebula:virtio: [name:neb
[ 7.957418][T1100068] kworker/ul6:2: [name:nebula_log_driver&]nebula: (0) [00008.9
```

2.2.1.3 uos 串口 log

1, uos early console

Android UOS early console 也会以 hypervisor log 形式输出，所以 log 还是以“nebula:”为开头

```

boot_linux_fdt:724: lk boot mode = 0
boot_linux_fdt:726: lk finished --> jump to linux kernel 64Bit

hello platform_quiesce
hello platform_quiesce done
[ 18.204947][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.207184][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.210739][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.213295][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.215190][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.217106][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.219163][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.221396][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.223235][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.225439][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.227320][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.230165][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.232913][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.235673][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.238387][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.241211][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.243980][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.246889][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.249712][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.252557][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.255457][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.258389][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]
[ 18.261374][T1400068] kworker/u16:2: [name:nebula_log_driver&]nebula: (2) [00028.2]

```

2, uos early console end

```

2 [ 20.192101] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 pci.cc:579]
3 [ 20.247103][T1400867] kworker/u16:4: [name:nebula_log_driver&]nebula: (5) [0003
4 [ 20.249249][T1400867] kworker/u16:4: [name:nebula_log_driver&]nebula: (5) [0003
5 [ 20.251302][T1400867] kworker/u16:4: [name:nebula_log_driver&]nebula: (5) [0003
6 [ 20.253473][T1400867] kworker/u16:4: [name:nebula_log_driver&]nebula: (5) [0003
7 [ 20.204778] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 io_server.c
8 [ 20.216097] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 pci.cc:579]
9 [ 20.268552][T1400867] kworker/u16:4: [name:nebula_log_driver&]nebula: (6) [0003
0 [ 20.270429][T1400867] kworker/u16:4: [name:nebula_log_driver&]nebula: (6) [0003
1 [ 20.228325] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 io_server.c
2 [ 20.240190] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 pci.cc:579]
3 [ 20.252125] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 io_server.c
4 [ 20.264205] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 pci.cc:579]
5 [ 20.276100] nbl_vm_srv[2705]: nbl_vmm I 12-02 18:15:21 2705 2751 io_server.c
6 [ 20.288138] nbl_vm_srv[2705]: [INFO:virtio_mmio.cc(214)] virtio-mmio device re
7 [ 20.300116] nbl_vm_srv[2705]: [INFO:virtio_pci.cc(566)] virtio PCI device rese
8 [ 20.312093] nbl_vm_srv[2705]: [INFO:virtio_pci.cc(566)] virtio PCI device rese

```

如上图，当看到“init: printk: console [hvc0] enabled” 就表示 early console 已结束，准备要切换到正常 console 设备

3, uos 串口日志

```

[ 20.520188] n[ 20.520286] nbl_vm_srv[2705]: [ 1.718356] [T600001] init: init
[ 20.520312] nbl_vm_srv[2705]: [ 1.720939] [T600001] init: init 7: Loaded kerne
[ 20.520329] nbl_vm_srv[2705]: [ 1.722787] [T600001] init: init 6: Loading modu
[ 20.520346] nbl_vm_srv[2705]: [ 1.725264] [T600001] init: init 6: Loaded kerne
[ 20.520361] nbl_vm_srv[2705]: [ 1.727021] [T600001] init: init 6: Loading modu
[ 20.520377] nbl_vm_srv[2705]: [ 1.729471] [T600001] init: init 6: Loaded kerne
[ 20.520393] nbl_vm_srv[2705]: [ 1.731513] [T600001] init: init 6: Loading modu
[ 20.520408] nbl_vm_srv[2705]: [ 1.733183] [T600001] init: [name:main&]virtio_e
[ 20.520423] nbl_vm_srv[2705]: [ 1.735820] [T400001] init: init 6: Loaded kerne
[ 20.520446] nbl_vm_srv[2705]: [ 1.737845] [T400001] init: init 6: Loading modu
[ 20.520461] nbl_vm_srv[2705]: [INFO:virtio_pci.cc(555)] virtio PCI device ready:
[ 20.520475] nbl_vm_srv[2705]: [ 1.759591] [T400001] init: virtio_apu virtiol6
[ 20.520490] nbl_vm_srv[2705]: [ 1.762256] [T400001] init: init 6: Loaded kerne
[ 20.520505] nbl_vm_srv[2705]: [ 1.764425] [T400001] init: init 6: Loading modu
[ 20.520522] nbl_vm_srv[2705]: [INFO:virtio_pci.cc(555)] virtio PCI device ready:
[ 20.520537] nbl_vm_srv[2705]: [ 1.771697] [T500131] virtio_init_kth: virtio-ei
[ 20.520552] nbl_vm_srv[2705]: [INFO:virtio_pci.cc(555)] virtio PCI device ready:
[ 20.520568] nbl_vm_srv[2705]: [ 1.788808] [T400001] init: init 7: Loaded kerne
[ 20.520583] nbl_vm_srv[2705]: [ 1.791405] [T400001] init: init 7: Loading modu

```

如上图所示，输出的 log 会以 “nbl_vm_srv[XXX]” 为开头，其中 XXX 是 nbl_vmm 的 pid 号。

最新关键词也可以使用 vmid:0 代表安卓，vmid:1 代表 TBOX

2.2.2 回读解析 expdb 分区

1，回读 expdb 分区



expdb.pdf

2，解析

python expdb_parser.pyc expdb(db) (需要 GAT 带的 python 和里面的脚本)

带路径方式

下载 GAT4.0 后，在路径 gat-win32-x86_64-4\plugin-scripts\expdb_parser 下有脚

本 expdb_parser.pyc，请使用

gat-win32-x86_64-4\prebuilt\python\bin\python.exe 执行 expdb_parser.pyc:

例如:

D:\gat-win32-x86_64-4\prebuilt\python\bin\python.exe D:\gat-win32-x86_64-

4\plugin-scripts\expdb_parser\expdb_parser.pyc expdb_全路径

```
PS C:\Users\PVT> D:\Software\GAT\GAT_exe_v4.2028.3\gat-win32-x86_64-4\prebuilt\python\bin\python.exe D:\Software\GAT\GAT_exe_v4.2028.3\gat-win32-x86_64-4\plugin-scripts\expdb_parser\expdb_parser.pyc "C:\Users\PVT\Documents\WeChat Files\wxid_bjnf5Subn9422\FileStorage\File\2024-10\ROM_0"
remove old C:\Users\PVT\Documents\WeChat Files\wxid_bjnf5Subn9422\FileStorage\File\2024-10\expdb_out first
-----start expdb parsing-----
C:\Users\PVT\Documents\WeChat Files\wxid_bjnf5Subn9422\FileStorage\File\2024-10\ROM_0
Head expdb header:
magic:0xdeadbeef, version:0x12, size:0x2000, datas:0x0, dhlk:0x0, blksize:0x1000, partsize:0x7a0000, bufsize:0x0, buf:0x0
type:0x0, valid:0x1, offset:0x3000, used:0x1000, total:0x1000, encrypt:0x0, raw:0x0, compact:0x0, name:b'SYS_RAMCONSOLE_RAW\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
=====
```

三，案例讲解

3.1 kernel 踩内存

本地自造问题，kasan 产生 db，gat 解析 db，通过 kernel log 按如下分析。

1, kasan 原理

编译插桩+影子内存+指针 tag

影子内存本身占据了一段内核虚拟地址空间。KASAN 的核心是建立一个从原内存地址到其对应的影子内存地址的映射关系。这个映射由一个统一的公式定义：

$$\text{shadow_addr} = (\text{addr} \gg \text{KASAN_SHADOW_SCALE_SHIFT}) +$$

$$\text{KASAN_SHADOW_OFFSET}$$

映射比例为 $2^{\text{KASAN_SHADOW_SCALE_SHIFT}}:1$ ，sw-tag 为 16:1。

对于 tag-based KASAN

- 0xFF 为默认的 kernel 内存 tag，即不启用 KASAN 的情况下，高 8bits 默认都是 0xFF
- 0xFE 表示不可访问的 memory，为了减少漏检，不对不可访问的 memory 继续细分占用 tag
- 0xFD 为随机 tag 的最大值，即 SW tag-based KASAN 在 0x00-0xFD 范围内随机取值；

2, kasan report

[671.734264] [T1418218] sh: licq buf:16, size:16, ptr=51ffff802ae2c9c0

行 31022: [671.736716] [T1418218] sh:

[name:report&]=====

=

行 31023: [671.736721] [T1418218] sh: [name:report&]BUG: KASAN: invalid-access in mysem_store+0xe8/0x130 [mysem]

行 31025: [671.739614] [T1418218] sh: [name:report&]Write of size 1 at addr 51ffff802ae2c9d0 by task sh/18218

行 31026: [671.739618] [T1418218] sh: [name:report_sw_tags&]Pointer tag: [51], memory tag: [fe]

行 31027: [671.739621] [T1418218] sh: [name:report&]

指针存储的 tag 和 影子内存 tag 不匹配。

3, kasan dump 附近内存数据

行 31100: [671.742890] [T1418218] sh: [name:report&]

行 31101: [671.742891] [T1418218] sh: [name:report&]Memory state around the buggy address:

行 31102: [671.742893] [T1418218] sh: fffffff802ae2c700: fe fe fe fe fe fe fe fe be fe fe fe fe fe fe fe

行 31103: [671.742895] [T1418218] sh: fffffff802ae2c800: fe fe fe fe a8 a8 fe fe fe fe fe fe fe fe fe

行 31105: [671.744969] [T1418218] sh: >ffffff802ae2c900: 46 46 46 46 fe fe fe fe fe fe fe fe 51 fe fe fe

行 31106: [671.744971] [T1418218] sh: [name:report&] ^

行 31108: [671.746522] [T1418218] sh: fffffff802ae2ca00: fe fe fe fe fe fe fe fe fe 95 95 95 95 fe fe fe fe

行 31109: [671.746524] [T1418218] sh: fffffff802ae2cb00: fe fe fe fe aa aa aa aa fe fe fe fe fe fe fe fe

行 31110: [671.746525] [T1418218] sh:

[name:report&]=====

Memory state around the buggy address:为访问数据附近 对应影子内存内存数据。

802ae2c9d0 对应影子内存内存为 51, 后续每 16 字节都为 fe。

4, kasan report 堆栈信息

行 31110: [671.746525] [T1418218] sh:

[name:report&]=====

=

行 31118: [671.763689] [T1418218] sh: [name:panic&]Disabling lock debugging due to kernel taint

行 31120: [671.766008] [T1418218] sh: [name:panic&]Kernel panic - not syncing: panic_on_taint set ...

行 31121: [671.766013] [T1418218] sh: CPU: 4 PID: 18218 Comm: sh Tainted: G S B W OE 6.1.124 #1

行 31122: [671.766018] [T1418218] sh: Hardware name: MT6897(ENG) (DT)

行 31123: [671.766020] [T1418218] sh: Call trace:

行 31124: [671.766022] [T1418218] sh: dump_backtrace+0xf8/0x148

行 31125: [671.766045] [T1418218] sh: show_stack+0x18/0x24

行 31126: [671.766049] [T1418218] sh: dump_stack_lvl+0x60/0x7c

行 31127: [671.766061] [T1418218] sh: dump_stack+0x18/0x38

行 31128: [671.766064] [T1418218] sh: panic+0x3a0/0xa18

```

行 31129: [ 671.766067] [T1418218] sh: add_taint+0x1a8/0x200
行 31130: [ 671.766072] [T1418218] sh: end_report+0x84/0x12c
行 31131: [ 671.766088] [T1418218] sh: kasan_report+0xe8/0x14c
行 31132: [ 671.766091] [T1418218] sh: kasan_tag_mismatch+0x28/0x40
行 31133: [ 671.766095] [T1418218] sh: __hwasan_tag_mismatch+0x2c/0x5c
行 31134: [ 671.766109] [T1418218] sh: mysem_store+0xe8/0x130 [mysem]
行 31135: [ 671.766126] [T1418218] sh: dev_attr_store+0x5c/0x80
行 31136: [ 671.766136] [T1418218] sh: sysfs_kf_write+0x154/0x1ac
行 31137: [ 671.766151] [T1418218] sh: kernfs_fop_write_iter+0x2fc/0x48c
行 31138: [ 671.766154] [T1418218] sh: vfs_write+0x628/0x820
行 31139: [ 671.766164] [T1418218] sh: ksys_write+0xe8/0x1b4
行 31140: [ 671.766167] [T1418218] sh: __arm64_sys_write+0x90/0xb8
行 31141: [ 671.766170] [T1418218] sh: invoke_syscall+0x88/0x274
行 31142: [ 671.766182] [T1418218] sh: el0_svc_common+0x154/0x280
行 31143: [ 671.766185] [T1418218] sh: do_el0_svc+0x58/0x11c
行 31144: [ 671.766188] [T1418218] sh: el0_svc+0x2c/0x58
行 31145: [ 671.766192] [T1418218] sh: el0t_64_sync_handler+0x68/0xb4
行 31146: [ 671.766195] [T1418218] sh: el0t_64_sync+0x198/0x19c

```

5, mysem_store 定位如下

```

97
98 static ssize_t mysem_store(struct device *dev,
99                          struct device_attribute *attr,
100                          const char *buf, size_t count)
101 {
102     char *ptr;
103     int size = 0;
104     int ret = 0;
105     int i = 0;
106
107     ret = kstrtoint(buf, 10, &size);
108     ptr = kmalloc(size, GFP_KERNEL);
109     printk("licq buf:%s, size:%d, ptr=%p\n", buf, size, ptr);
110     for (i=0; i <= size; i++) {
111         ptr[i] = 'a';
112     }
113
114     return count;
115 }

```

ptr 存在越界访问, kmalloc 返回的地址是 51ffff802ae2c9c0, 问题报错在 51ffff802ae2c9d0。

3.2 user 版本 Android 无法启动

1, 问题现象

user 版本 Android 无法启动，屏不亮

2，问题分析

1) 串口 log 显示 Android 发生 KE

```
39.482760] nbl_vm_srv[4232]: nbl_vmm vmmid:0 W 09-16 13:51:49 4232 4342 vmctl.cc:77] Host vmm recieve VMCTL KERNEL PANIC
39.489508] nbl_vm_srv[4232]: nbl_vmm vmmid:0 E 09-16 13:51:49 4232 4361 lk2.cc:130] set exp type failed: it had been set as 2 before
39.501792] nbl_vm_srv[4227]: [2025-09-16T05:51:49Z INFO virtiofsd] Client disconnected, shutting down
39.504854] nbl_vm_srv[4226]: [2025-09-16T05:51:49Z INFO virtiofsd] Client disconnected, shutting down
39.510438] nbl_vm_srv[4229]: [2025-09-16T05:51:49Z INFO virtiofsd] Client disconnected, shutting down
39.524754] nbl_vm_srv[4230]: [2025-09-16T05:51:49Z INFO virtiofsd] Client disconnected, shutting down
```

2) 回读 android 端 expdb 分区

Android 一直发生 KE，无法正常导出 db 文件，采用回读 android 端 expdb 分区进行分析
第一次回读 last_kernel 日志 size 为 0，无 log。分析 pl_lk 判断 loglevel 为 0 导致

```
2192 get_suffix:205: [LK_AB] metadata.slot_info.tbox.successful = 1, 0
2193 android_ab_ota_boot_check:904: [LK_AB] android current ab_suffix: _a, android_ab_retry: 7
2194 android_ab_ota_boot_check:907: [LK_AB] _a android boot_success is 1!
2195 target init
2196 defined(VARIANT_USER):loglevel=0
2197 wff yocto lk2 kedump: last is not full pmic reset!
2198 kedump: normal flow
2199 kedump: found content in expdb
2200 kedump: read SYS_PSTORE_RAW from offset 0x4000 size 0xe0000
2201 kedump: read SYS_RAMCONSOLE_RAW from offset 0x3000 size 0x1000
```

3) Lk loglevel 更改为 7

304535 [ISSUE ID#12345][ISSUE SUMMARY]User version loglevel too low [TYPE]Task [PROJECT]8678 [MODULE]Baseline Update [DESCRIPTION]improve user version loglevel from 0 to 7 [COMPANY]邦德垣 [TEAM]数字科学_智能运维_BSP platform/mediatek/common/debug/dynamic_log.c

```
80 memset(tpbuf, 0x0, sizeof(tpbuf));
81 sprintf(tpbuf, sizeof(tpbuf) - 1, "console%04d,%d", log_port(), log_baudrate());
82 kcmdline_append(tpbuf);
83 #endif
84
85 memset(tpbuf, 0x0, sizeof(tpbuf));
86 sprintf(tpbuf, sizeof(tpbuf) - 1, "log_buf_len=%s", PRINTK_BUFFER_SIZE);
87 kcmdline_append(tpbuf);
88 kcmdline_append("mk_printk_ctrl.disable_uart=1");
89 kcmdline_append("loglevel=7");
90 printf("defined(VARIANT_USER):loglevel=%d\n", loglevel);
91
92
93
94
95 LK_INIT_HOOK(dynamic_log, dynamic_log_set_cmdline, LK_INIT_LEVEL_TARGET);
96
97 #if 0
98 static void log_com_pl_boottag_hook(struct boot_tag *tag)
99 {
100     memcpy(&log_com_state, tag->data, sizeof(log_com_state));
101     PL_BOOTTAGS_INIT_HOOK(log_com, BOOT_TAG_L00.COM, log_com_pl_boottag_hook);
102 #endif
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

4) 第二次回读 expdb 分区

last_kmsg 显示 Unknown symbol pid_vnr

完整异常 log:

```
[ 2.488204][ T1] carevent_wt: Unknown symbol pid_vnr (err -2)
[ 2.489579][ T1] Kernel panic - not syncing: Attempted to kill init!
exitcode=0x00007f00
```

内部经验：这是 carevent_wt 不符合 gki 标准导致的。

3，解决方案

1) 关闭 gki

下面标注位置改成 no，关闭 gki

```
xcfe: /pvt_8678_305/MT8678_305/device/mediatek/vendor/common/BoardConfig-vext.mk
Home | History | Annotate | Line# | Scopes# | Navigate# | Raw | Download | Search | current directory

120
121 #Add MTK's Recovery fstab definitions
122 TARGET_RECOVERY_FSTAB := $(MTK_PUGEN_PRODUCT_OUT)/$(TARGET_COPY_OUT_VENDOR)/etc/fstab.$(TARGET_BOARD_PLATFORM)
123
124 BOARD_RAMDISK_USE_L24 := true
125 ifndef MTK_USE_PREBUILT_BOOTIMAGE
126 ifneq ($(filter usb,$(TARGET_BUILD_VARIANT)),)
127 MTK_USE_PREBUILT_BOOTIMAGE := yes
128 endif
129 endif
130 ifeq ($(MTK_USE_PREBUILT_BOOTIMAGE),yes)
131 MTK_BOARD_PREBUILT_BOOTIMAGE := $(wildcard vendor/aosp_gki/$(LINUX_KERNEL_VERSION)/aarch64/boot.img)
132 ifneq ($(MTK_BOARD_PREBUILT_BOOTIMAGE),)
133 ifeq ($(BOARD_EXCLUDE_KERNEL_FROM_RECOVERY_IMAGE),true)
134 TARGET_DO_KERNEL := true
135 endif
136 BOARD_PREBUILT_BOOTIMAGE := $(MTK_BOARD_PREBUILT_BOOTIMAGE)
137 endif
138 endif
139
140 ifndef HAL_TARGET_PROJECT
141 ALLOW_MISSING_DEPENDENCIES := true
142 BUILD_BROKEN_DUP_RULES := true
143 BUILD_BROKEN_USES_NETWORK := true
144 BUILD_BROKEN_VINTF_PRODUCT_COPY_FILES := true
145 ifneq ($(wildcard vendor/mediatek/internal),)
146 ifneq ($(COVERITY_LOCAL_SCAN),yes)
```

2) 更改 carevent_wt 以符合 gki 标准，客户完成修改。